OS Unit 4 Orange Problem

Name: Ananya Ratnaparkhi

SRN: PES2UG24CS058

Phase 1

1A: Output of ./test_objects showing all tests passing.

```
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B: find .pes/objects -type f showing the sharded directory structure.

```
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2

2A: Output of ./test_tree showing all tests passing.

```
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % find .pes/objects -type f
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % xxd .pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d | head -20
00000000: 626c 6f62 2031 3200 6865 6c6c 6f20 776f  blob 12.hello wo
00000010: 726c 640a                                rld.
```

Phase 3

3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  object.c
  untracked:  pes.c
  untracked:  Makefile
  untracked:  test_objects
  untracked:  tree.h
  untracked:  test_objects.c
  untracked:  commit.c
  untracked:  test_tree
  untracked:  README.md
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  test_tree.c
  untracked:  pes.h
  untracked:  commit.h
  untracked:  tree.c
```

3B: cat .pes/index showing the text-format index with your entries.

```
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % cat .pes/index
100644 0bd69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d 1776357354 12 file1.txt
100644 96c18f0297e38d01f4b2dacddea4259aea6b2961eb0822bd2c0c3f6029030045 1776357662 3 file2.txt
```

Phase 4

4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % ./pes log
commit 9593397d7d7e471b714e82d40b5b3fbb6b3245d2542f15382a14be213a6dc04c
Author: Ananya <PES2UG24CS058>
Date:   1776357992

    Add farewell

commit 03be0d458fe37788d1be27eab0dfce8f657d3f8d112f4ea0085daceb36b729d9
Author: Ananya <PES2UG24CS058>
Date:   1776357974

    Add world

commit d8e5a63392467600fe17aa9d7ddf74d76b647b9101ee4befa4a1584d70b0cb50
Author: Ananya <PES2UG24CS058>
Date:   1776357957

    Initial commit
```

4B: find .pes -type f | sort showing object store growth after three commits.

```
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/03/be0d458fe37788d1be27eab0dfce8f657d3f8d112f4ea0085daceb36b729d9
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/23/e9df7deeb2df941f4e9136eadea050a096fd54cc99d43cd857be049351246c
.pes/objects/60/96ac644b35b0a4e3aca12b7d97f19dc4013f6d5247e2de4ede71cde6e45188
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/66/9b7500487df0854b1772f597b02df18244b2c30a4cedab1a234c583c7d84aa
.pes/objects/95/93397d7d7e471b714e82d40b5b3fbb6b3245d2542f15382a14be213a6dc04c
.pes/objects/96/c18f0297e38d01f4b2dacddea4259aea6b2961eb0822bd2c0c3f6029030045
.pes/objects/d8/e5a63392467600fe17aa9d7ddf74d76b647b9101ee4befa4a1584d70b0cb50
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % cat .pes/refs/heads/main
9593397d7d7e471b714e82d40b5b3fbb6b3245d2542f15382a14be213a6dc04c
Ananya@Nikhils-MacBook-Air-2 PES2UG24CS058-pes-vcs % cat .pes/HEAD
ref: refs/heads/main
```

Phase 5 & 6: Analysis-Only Questions

<u>Branching and Checkout</u>

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?
Ans: To implement pes checkout <branch>, you would update .pes/HEAD to point to the selected branch reference and ensure .pes/refs/heads/<branch> contains the correct commit hash. Then you would rebuild the index from that commit's tree and update the working directory to match it by writing files, updating contents, and deleting files that don't exist in the target branch. The complexity comes from needing to keep HEAD, index, and working directory consistent while also handling conflicts and avoiding partial updates that could corrupt state.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.
Ans: You detect a dirty working directory by comparing three versions of each file: the working directory, the index, and the target branch's commit tree. If a file differs between working directory and index (unstaged change) and also differs between index and target commit (would be overwritten), checkout must fail. This uses the index to represent the last known clean state and the object store to get the target branch contents for comparison.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?
Ans: In detached HEAD, new commits are created normally but are not attached to any branch, so they can become unreachable when you switch away. You can recover them using the commit hash (via logs or reflog-like history) and then creating a new branch pointing to that commit, which makes the commits reachable again.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.
Ans: First, start from all reachable references (branch tips, HEAD, tags) and use DFS or BFS from those roots to mark all reachable commits, trees, and blobs in a set. Finally, sweep the object store and delete anything that isn't reachable (not found in the set).

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?
Ans: Concurrent GC and commit is dangerous because GC might not see a newly created object during marking and delete it during sweep before it gets referenced, causing missing history. Git avoids this using atomic updates, locking of refs, and treating object writes and reference updates in a way that ensures objects are fully written and safely reachable before GC can remove anything.